



Atividade Guiada: Integração com APIs

Atividade Guiada: Integração com APIs

GeoBusca CEP: Construindo um App com React, APIs, Material-UI e Styled-Components

João Marcelo de Almeida Garcia

16 de setembro de 2026

Entrega da atividade

A atividade deverá ser realizada da seguinte forma:

A atividade deverá ser disponibilizada no GitHub. Para isso, crie um repositório com o seguinte nome:

`atividade-guiada-aula-05-frameworks-1`

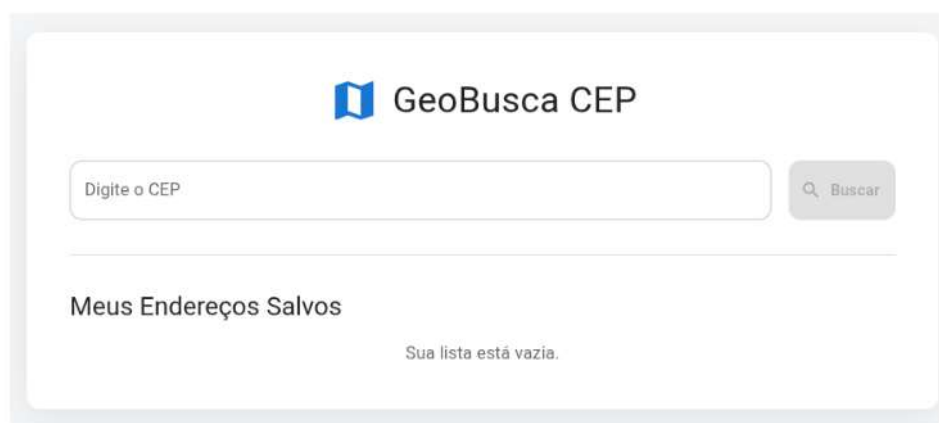
Após concluir todas as atividades, faça o upload de todos os arquivos para o repositório do GitHub.

Ao finalizar, compartilhe o link do seu repositório no campo de resposta da atividade.

Atenção: o repositório deverá estar configurado como **público**, para que seja possível acessar e avaliar a atividade.

O Resultado Que Você Vai Construir

Ao final deste guia, você terá uma aplicação completamente funcional como a demonstrada abaixo, capaz de buscar endereços, exibi-los em um mapa e salvar seus locais favoritos.



Passo 0: Preparando o Ambiente

Vamos construir uma aplicação que busca um endereço por CEP, exibe no mapa e permite salvar favoritos. Para começar, abra seu terminal e execute os comandos para criar e configurar o projeto React com Vite:

```
# 1. Crie o projeto React com Vite
npm create vite@latest geobusca-cep -- --template react
# 2. Navegue para a pasta do projeto
cd geobusca-cep
# 3. Instale todas as dependências necessárias
npm install axios styled-components @mui/material @emotion/react @emotion/styled
  ↳ @mui/icons-material react-leaflet leaflet
# 4. Inicie o servidor de desenvolvimento
npm run dev
```

Passo 1: Obtendo a Chave da API do OpenCage

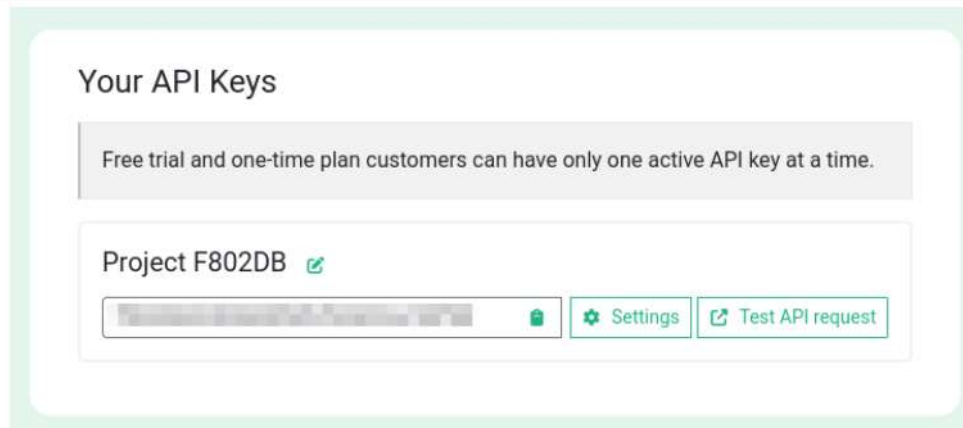
Objetivo da Etapa

Para que nossa aplicação consiga converter um endereço em coordenadas para o mapa, precisamos de uma chave de API (um tipo de "senha") do serviço OpenCage. O processo é gratuito e rápido.

1. **Acesse o site:** Abra seu navegador e vá para opencagedata.com.
2. **Crie sua conta:** Clique em "Start your free trial". Você pode se registrar usando seu e-mail ou uma conta do Google/GitHub. O plano gratuito é mais que suficiente para nosso projeto.
3. **Acesse o Painel (Dashboard):** Após o registro e login, você será redirecionado para o seu painel de controle.
4. Clique na opção abaixo



5. **Copie sua Chave:** Na seção "API Keys", você verá uma chave já gerada para você. Clique no botão de copiar ao lado dela.



Sua chave de API estará localizada nesta seção do painel.

Passo 2: Estrutura, Tema e Estilo Global

Com o projeto criado, vamos definir a identidade visual da nossa aplicação. Criaremos um arquivo de tema centralizado que será usado por todos os nossos componentes, garantindo uma UI coesa e profissional.

Arquivo src/theme.js

```
// src/theme.js
import { createTheme } from '@mui/material/styles';

const theme = createTheme({
  palette: {
    primary: { main: '#1976d2' },
    secondary: { main: '#9c27b0' },
    background: { default: '#f4f6f8', paper: 'ffffff' },
  },
  typography: { fontFamily: '"Roboto", "Helvetica", "Arial", sans-serif' },
  shape: { borderRadius: 12 },
  components: {
    MuiButton: {
      styleOverrides: {
        root: { textTransform: 'none', padding: '10px 20px' },
      },
    },
  },
});
export default theme;
```

Passo 3: Conectando com as APIs

Agora vamos criar os módulos responsáveis por se comunicar com as APIs externas. Isolar essa lógica em arquivos separados (na pasta 'src/api') é uma excelente prática de organização.

Arquivo .env (Na Raiz do Projeto)

```
# .env (na raiz do projeto)
VITE_OPENCAGE_API_KEY=SUA_CHAVE_API_QUE_VOCE_COPIOU_NO_PASSO_1
```

Arquivo src/api/viaCep.js

```
// src/api/viaCep.js
import axios from 'axios';

const viaCepApi = axios.create({
  baseURL: 'https://viacep.com.br/ws',
});

export const getAddressByCep = async (cep) => {
  const response = await viaCepApi.get(`/${cep}/json/`);
  if (response.data.erro) {
    throw new Error('CEP não encontrado.');
```

Arquivo src/api/opencage.js

```
// src/api/opencage.js
import axios from 'axios';

const API_KEY = import.meta.env.VITE_OPENCAGE_API_KEY;

const opencageApi = axios.create({
  baseURL: 'https://api.opencagedata.com/geocode/v1/json',
});

export const getCoordinatesByAddress = async (address) => {
  const query = `${address.logradouro}, ${address.localidade}, ${address.uf},
  ↪ Brasil`;

  const response = await opencageApi.get('', {
    params: { q: query, key: API_KEY, limit: 1, language: 'pt-BR' },
  });

  if (response.data.results.length === 0) {
    throw new Error('Coordenadas não encontradas.');
```


Passo 4: Criando os Componentes da Interface

Objetivos da Etapa

Vamos construir as "peças de Lego" da nossa interface. Cada componente terá uma responsabilidade única (Single Responsibility Principle), tornando nosso código mais limpo, reutilizável e fácil de manter.

Componente: CepForm.jsx

```
// src/components/CepForm.jsx
import React, { useState } from "react";
import { TextField, Button, Box } from "@mui/material";
import { Search } from "@mui/icons-material";

const CepForm = ({ onSearch, loading }) => {
  const [cep, setCep] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();
    const cleanedCep = cep.replace(/\D/g, "");
    if (cleanedCep.length === 8) onSearch(cleanedCep);
  };

  return (
    <Box
      component="form"
      onSubmit={handleSubmit}
      sx={{ mt: 4, display: "flex", gap: 2 }}
    >
      <TextField
        label="Digite o CEP"
        variant="outlined"
        fullWidth
        value={cep}
        onChange={(e) => setCep(e.target.value)}
      />
      <Button
        type="submit"
        variant="contained"
        disabled={loading || cep.replace(/\D/g, "").length !== 8}
        startIcon={<Search />}
      >
        {loading ? "Buscando..." : "Buscar"}
      </Button>
    </Box>
  );
};

export default CepForm;
```

Visão Geral do Componente

A única responsabilidade do 'CepForm' é gerenciar o campo de input e informar o componente pai ('App.jsx') quando uma busca deve ser iniciada. Ele não sabe como a busca funciona, apenas delega a ação através da prop 'onSearch'.

Estilo e CSS

A estilização é feita com a prop `sx` do Material-UI, que permite escrever CSS diretamente no componente:

- `mt: 4`: Adiciona uma margem no topo (`margin-top`) de 4 unidades de espaçamento do tema (32px).
- `display: 'flex'`: Transforma o `<Box>` em um container flexbox, alinhando seus filhos (o campo de texto e o botão) horizontalmente.
- `gap: 2`: Cria um espaçamento (`gap`) de 2 unidades (16px) entre os itens dentro do flexbox.

Conceitos Aplicados

- **Hook 'useState'**: Usado para criar uma "variável de estado"(`cep`) que armazena o valor do campo de texto. Quando `setCep` é chamado, o componente re-renderiza com o novo valor.
- **Componente Controlado**: O valor do `TextField` é diretamente controlado pelo estado `cep` do React (`value={cep}`). Isso garante que o React seja a única fonte da verdade para o que é exibido no campo.
- **Comunicação via Props**: A função `onSearch` é passada do `App.jsx` para o `CepForm`. Ao submeter o formulário, `CepForm` "avisa" o pai executando essa função.

Componente: AddressDisplay.jsx

```
// src/components/AddressDisplay.jsx
import React from "react";
import {
  Card,
  CardContent,
  Typography,
  Grid,
  Button,
  Box,
  Slide,
  Chip,
} from "@mui/material";
import { Save, Close, Phone } from "@mui/icons-material";

const AddressDisplay = ({ address, isFavoriteView, onSave, onClose }) => {
  return (
    <Slide direction="down" in={true} mountOnEnter unmountOnExit>
```

```

<Card
  sx={{
    mt: 4,
    mb: 2,
    backgroundColor: "#f9f9f9",
    border: "1px solid #e0e0e0",
  }}
>
<CardContent>
  <Box
    sx={{
      display: "flex",
      justifyContent: "space-between",
      alignItems: "center",
      mb: 2,
    }}
  >
    <Typography variant="h6" color="primary.dark">
      {isFavoriteView ? address.apelido : "Endereço Encontrado"}
    </Typography>
    {isFavoriteView ? (
      <Button
        variant="outlined"
        startIcon={<Close />}
        onClick={onClose}
        color="secondary"
      >
        Fechar
      </Button>
    ) : (
      <Button variant="contained" startIcon={<Save />} onClick={onSave}>
        Salvar Favorito
      </Button>
    )}
  </Box>
  <Grid container spacing={2}>
    <Grid item xs={12}>
      <Typography>
        <strong>Logradouro:</strong> {address.logradouro || "N/A"}
      </Typography>
    </Grid>
    <Grid item xs={12} sm={8}>
      <Typography>
        <strong>Cidade:</strong> {address.localidade}
      </Typography>
    </Grid>
    <Grid item xs={12} sm={4}>
      <Typography>
        <strong>UF:</strong> {address.uf}
      </Typography>
    </Grid>
    <Grid item xs={12} sm={8}>

```



```

    <Typography>
      <strong>Bairro:</strong> {address.bairro}
    </Typography>
  </Grid>
  <Grid item xs={12} sm={4}>
    {address.ddd && (
      <Chip
        icon={<Phone />}
        label={`DDD: ${address.ddd}`}
        variant="outlined"
        color="primary"
        size="small"
      />
    )}
  </Grid>
</Grid>
</CardContent>
</Card>
</Slide>
);
};
export default AddressDisplay;

```

Visão Geral do Componente

Este é um "componente de apresentação". Sua única função é receber dados (`address`) e funções (`onSave`, `onClose`) via props e exibi-los de forma organizada.

Estilo e CSS

- `<Card>`: `mt: 4` e `mb: 2` adicionam margens verticais. `backgroundColor` e `border` personalizam a aparência do card para destacá-lo do fundo.
- `<Box>`: Novamente, usamos `display: 'flex'` para alinhar o título e o botão na mesma linha. `justifyContent: 'space-between'` os empurra para as extremidades opostas e `alignItems: 'center'` os alinha verticalmente.

Conceitos Aplicados

- **Props Destructuring**: A sintaxe (`{ address, isFavoriteView, ... }`) na declaração da função permite extrair as propriedades do objeto props diretamente em variáveis, deixando o código mais limpo.
- **Renderização Condicional**: O operador ternário `{isFavoriteView ? <Button /> : <Button />}` é usado para exibir um botão diferente (Fechar ou Salvar) com base no valor da prop `isFavoriteView`.

Componente: MapView.jsx

```

// src/components/MapView.jsx
import React from "react";

```

```
import { MapContainer, TileLayer, Marker, Popup } from "react-leaflet";
import { Paper, Fade } from "@mui/material";
import "leaflet/dist/leaflet.css";

const MapView = ({ coordinates }) => {
  const mapKey = `${coordinates.lat}-${coordinates.lng}`;

  return (
    <Fade in={true} timeout={500}>
      <Paper
        elevation={6}
        sx={{ height: "400px", width: "100%", overflow: "hidden", mt: 2 }}
      >
        <MapContainer
          key={mapKey}
          center={[coordinates.lat, coordinates.lng]}
          zoom={17}
          style={{ height: "100%", width: "100%" }}
        >
          <TileLayer
            url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png"
            attribution='© <a
              ↳ href="https://www.openstreetmap.org/copyright">OpenStreetMap</a>
              ↳ contributors'
          />
          <Marker position={[coordinates.lat, coordinates.lng]}>
            <Popup>Localização do endereço.</Popup>
          </Marker>
        </MapContainer>
      </Paper>
    </Fade>
  );
};

export default MapView;
```

Visão Geral do Componente

Este componente é um "wrapper" para a biblioteca `react-leaflet`. Sua função é receber as `coordinates` via props e renderizar um mapa (`<MapContainer>`), um provedor de imagens (`<TileLayer>`) e um marcador (`<Marker>`) na localização exata.

Estilo e CSS

- `<Paper>`: O componente `<Paper>` do MUI é usado como um container para o mapa, dando a ele uma elevação (sombra) e bordas arredondadas.
- `height: '400px'`: Define uma altura fixa para o container do mapa.
- `width: '100%'`: Faz o mapa ocupar toda a largura disponível.
- `overflow: 'hidden'`: Essencial para garantir que o mapa, que é renderizado dentro do Paper, não "vaze" para fora das bordas arredondadas.

Conceitos Aplicados

- **Importação de CSS:** A linha `import 'leaflet/dist/leaflet.css';` é crucial. Muitas bibliotecas de componentes vêm com seu próprio CSS para estilização, que precisa ser importado para que elas funcionem visualmente.
- **Prop 'key' Dinâmica:** A prop `key` é uma ferramenta poderosa do React. Quando a `key` de um componente muda, o React o destrói e o recria do zero. Usamos isso aqui para forçar o `<MapContainer>` a ser totalmente recarregado com as novas coordenadas centrais a cada nova busca.

Componente: FavoritesList.jsx

```
// src/components/FavoritesList.jsx
import React, { useState } from "react";
import {
  Typography,
  List,
  ListItem,
  ListItemText,
  IconButton,
  TextField,
  Box,
  ListItemButton,
  ListItemAvatar,
  Avatar,
  Collapse,
} from "@mui/material";
import { Edit, Delete, Save, Cancel, LocationOn } from "@mui/icons-material";
import { TransitionGroup } from "react-transition-group";

const FavoritesList = ({ favorites, onUpdate, onDelete, onSelect }) => {
  const [editingId, setEditingId] = useState(null);
  const [editText, setEditText] = useState("");

  const handleEditStart = (favorite) => {
    setEditingId(favorite.id);
    setEditText(favorite.apelido);
  };

  const handleEditCancel = () => {
    setEditingId(null);
    setEditText("");
  };

  const handleEditSave = () => {
    if (editText.trim()) {
      onUpdate(editingId, editText.trim());
    }
    handleEditCancel();
  };
};
```

```

return (
  <Box sx={{ mt: 2 }}>
    <Typography variant="h5" component="h2" gutterBottom>
      Meus Endereços Salvos
    </Typography>
    {favorites.length === 0 ? (
      <Typography color="textSecondary" sx={{ mt: 2, textAlign: "center" }}>
        Sua lista está vazia.
      </Typography>
    ) : (
      <List>
        <TransitionGroup>
          {favorites.map((fav) => (
            <Collapse key={fav.id}>
              <ListItem divider disablePadding>
                {editingId === fav.id ? (
                  <Box sx={{ display: "flex", width: "100%", p: 2, gap: 1 }}>
                    <TextField
                      value={editText}
                      onChange={(e) => setEditText(e.target.value)}
                      fullWidth
                      autoFocus
                    />
                    <IconButton onClick={handleEditSave} color="primary">
                      <Save />
                    </IconButton>
                    <IconButton onClick={handleEditCancel}>
                      <Cancel />
                    </IconButton>
                  </Box>
                ) : (
                  <ListItemButton
                    onClick={() => onSelect(fav)}
                    sx={{ py: 2 }}
                  >
                    <ListItemAvatar>
                      <Avatar sx={{ bgcolor: "secondary.main" }}>
                        <LocationOn />
                      </Avatar>
                    </ListItemAvatar>
                    <ListItemText
                      primary={fav.apelido}
                      secondary={` ${fav.logradouro},
                        ↪  ${fav.localidade}/${fav.uf}`}
                    />
                    <IconButton
                      edge="end"
                      onClick={(e) => {
                        e.stopPropagation();
                        handleEditStart(fav);
                      }}
                    >

```



```

        <Edit />
      </IconButton>
      <IconButton
        edge="end"
        onClick={(e) => {
          e.stopPropagation();
          onDelete(fav.id);
        }}
      >
        <Delete />
      </IconButton>
    </ListItemButton>
  )}
</ListItem>
</Collapse>
)}}
</TransitionGroup>
</List>
)}
</Box>
);
};
export default FavoritesList;

```

Visão Geral do Componente

Este é o componente mais complexo até agora. Ele tem uma dupla responsabilidade: exibir a lista de **favorites** (recebida via props) e também gerenciar o estado da interface de edição (**editingId** e **editText**), que é um estado local, pertencente apenas a este componente.

Estilo e CSS

- **<ListItemButton>**: **py: 2** adiciona um preenchimento vertical (**padding-top** e **padding-bottom**), tornando os itens da lista mais altos e fáceis de clicar.
- **<Avatar>**: **bgcolor: 'secondary.main'** usa a cor secundária definida no nosso 'theme.js' como cor de fundo para o avatar, mantendo a consistência visual.
- **<Typography>**: **textAlign: 'center'** é usado para centralizar a mensagem "Sua lista está vazia".

Conceitos Aplicados

- **"Lifting State Up" (Elevar o Estado)**: A lista **favorites** não é gerenciada aqui. Ela "vive" no **App.jsx**. Este componente apenas a recebe e, quando precisa modificá-la (**update**, **delete**), ele chama as funções **onUpdate** e **onDelete** que recebeu do pai.
- **Renderização de Listas ('.map()')**: O método **.map()** é usado para transformar cada objeto do array **favorites** em um elemento JSX (**<Collapse>**). A prop

`key={fav.id}` é essencial para o React otimizar a renderização da lista.

- `'e.stopPropagation()'`: Eventos em HTML "borbulham" para cima. Sem isso, clicar no ícone de editar/deletar também acionaria o `onClick` do item da lista (`<ListItemButton>`). Esta função impede essa propagação.

Passo 5: Orquestrando o App.jsx

Objetivos da Etapa

Este é o "cérebro" da aplicação. Vamos juntar todas as peças: gerenciar o estado global com `useState`, lidar com efeitos colaterais (como salvar dados) com `useEffect`, orquestrar as chamadas de API e passar todos os dados e funções para os componentes filhos através de props.

```
// src/App.jsx
import React, { useState, useEffect } from "react";
import styled, {
  ThemeProvider as StyledThemeProvider,
} from "styled-components";
import {
  Container,
  Typography,
  CircularProgress,
  Alert,
  Divider,
  ThemeProvider as MuiThemeProvider,
  CssBaseline,
  Box,
} from "@mui/material";
import { Map } from "@mui/icons-material";
import theme from "../theme";
import { getAddressByCep } from "../api/viaCep";
import { getCoordinatesByAddress } from "../api/opencage";
import CepForm from "../components/CepForm";
import AddressDisplay from "../components/AddressDisplay";
import MapView from "../components/MapView";
import FavoritesList from "../components/FavoritesList";

const AppWrapper = styled.div`
  background-color: ${({ theme }) => theme.palette.background.default};
  min-height: 100vh;
  padding: 3rem 1rem;
`;

const MainContent = styled(Box)`
  background: ${({ theme }) => theme.palette.background.paper};
  padding: 2.5rem;
  border-radius: ${({ theme }) => theme.shape.borderRadius}px;
  box-shadow: 0 10px 40px rgba(0, 0, 0, 0.08);
`;
```

```
function App() {
  const [address, setAddress] = useState(null);
  const [coordinates, setCoordinates] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");
  const [isFavoriteView, setIsFavoriteView] = useState(false);

  const [favorites, setFavorites] = useState(() => {
    try {
      const savedFavorites = localStorage.getItem("favorites");
      return savedFavorites ? JSON.parse(savedFavorites) : [];
    } catch (error) {
      console.error("Failed to parse favorites from localStorage", error);
      return [];
    }
  });

  useEffect(() => {
    localStorage.setItem("favorites", JSON.stringify(favorites));
  }, [favorites]);

  const handleSearch = async (cep) => {
    setIsFavoriteView(false);
    setLoading(true);
    setError("");
    setAddress(null);
    setCoordinates(null);

    try {
      const addressData = await getAddressByCep(cep);
      setAddress(addressData);
      const coords = await getCoordinatesByAddress(addressData);
      setCoordinates(coords);
    } catch (err) {
      setError(err.message || "Ocorreu um erro inesperado.");
    } finally {
      setLoading(false);
    }
  };

  const handleAddFavorite = (currentAddress, currentCoordinates) => {
    const isAlreadyFavorite = favorites.some(
      (fav) => fav.cep === currentAddress.cep,
    );
    if (isAlreadyFavorite) {
      setError("Este endereço já está na sua lista de favoritos.");
      return;
    }
    const newFavorite = {
      id: Date.now(),
      apelido: "Novo Endereço",
    };
  };
}
```

```

    ...currentAddress,
    coordinates: currentCoordinates,
  };
  setFavorites((prevFavorites) => [newFavorite, ...prevFavorites]);
  handleCloseDisplay();
  setError("");
};

const handleUpdateFavorite = (id, newApelido) => {
  setFavorites((prevFavorites) =>
    prevFavorites.map((fav) =>
      fav.id === id ? { ...fav, apelido: newApelido } : fav,
    ),
  );
};

const handleDeleteFavorite = (id) => {
  setFavorites((prevFavorites) =>
    prevFavorites.filter((fav) => fav.id !== id),
  );
};

const handleSelectFavorite = (favorite) => {
  setAddress(favorite);
  setIsFavoriteView(true);
  setCoordinates(favorite.coordinates || null);
};

const handleCloseDisplay = () => {
  setAddress(null);
  setCoordinates(null);
  setIsFavoriteView(false);
};

return (
  <MuiThemeProvider theme={theme}>
    <StyledThemeProvider theme={theme}>
      <CssBaseline />
      <AppWrapper>
        <Container maxWidth="md">
          <MainContent>
            <Box
              sx={{
                display: "flex",
                alignItems: "center",
                justifyContent: "center",
                mb: 2,
              }}
            >
              <Map color="primary" sx={{ fontSize: 48, mr: 1.5 }} />
              <Typography variant="h4" component="h1">
                GeoBusca CEP
              </Typography>
            </Box>
          </MainContent>
        </Container>
      </AppWrapper>
    </StyledThemeProvider>
  </MuiThemeProvider>
);

```

```

    </Typography>
  </Box>
  <CepForm onSearch={handleSearch} loading={loading} />
  {loading && (
    <Box sx={{ display: "flex", justifyContent: "center", my: 3 }}>
      <CircularProgress />
    </Box>
  )}
  {error && (
    <Alert
      severity="error"
      sx={{ mt: 2 }}
      onClose={() => setError("")}
    >
      {error}
    </Alert>
  )}
  {address && (
    <AddressDisplay
      address={address}
      isFavoriteView={isFavoriteView}
      onSave={() => handleAddFavorite(address, coordinates)}
      onClose={handleCloseDisplay}
    />
  )}
  {coordinates && <MapView coordinates={coordinates} />}
  <Divider sx={{ my: 4 }} />
  <FavoritesList
    favorites={favorites}
    onUpdate={handleUpdateFavorite}
    onDelete={handleDeleteFavorite}
    onSelect={handleSelectFavorite}
  />
</MainContent>
</Container>
</AppWrapper>
</StyledThemeProvider>
</MuiThemeProvider>
);
}
export default App;

```

Estilo e CSS com Styled Components

Aqui usamos a biblioteca `styled-components` para criar componentes React com estilos CSS encapsulados. É uma alternativa à prop `sx`.

- **AppWrapper:** É um `<div>` estilizado que serve como o fundo principal da página. A função `${({ theme }) => ...}` permite acessar as cores que definimos no nosso `'theme.js'`, como `theme.palette.background.default`. Também definimos uma altura mínima de `100vh` (100% da altura da tela) e um `padding` geral.

- **MainContent:** É um componente `<Box>` do MUI estilizado. Ele cria o "card" principal branco (`theme.palette.background.paper`) com padding interno, bordas arredondadas (`border-radius`) vindas do tema e uma sombra sutil (`box-shadow`).

Que tal deixar uma avaliação da aula de hoje :)



<https://forms.cloud.microsoft/r/kXtNMBn0sA>